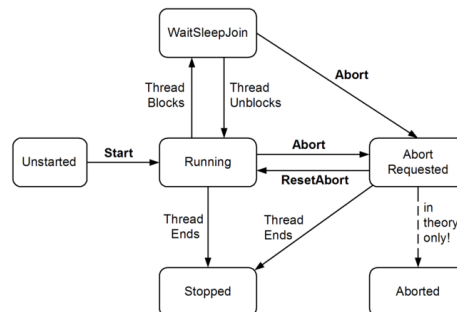




Multithreading

Multithreading

- Ziel: mehrere Handlungsstränge „parallel“ abarbeiten
- Es kann immer nur 1 Thread pro Kern gleichzeitig ausgeführt werden



Thread mit parameterloser Methode

```
public static void Main(string[] args)
{
    Thread t = new Thread(MachEtwas);
    t.Start();

    Console.ReadKey();
}

private static void MachEtwas()
{
    // Logik
}
```

Threads und Parameter

```
public static void Main(string[] args)
{
    Thread t = new Thread(new ParameterizedThreadStart(Berechne));
    t.Start(42);

    Console.ReadKey();
}

private static void Berechne(object c)
{
    // komplizierte Berechnung im Thread
}
```

Auf einen Thread warten

```
public static void Main(string[] args)
{
    Thread t = new Thread(MachEtwas);
    t.Start();

    t.Join(); // Warten, bis der Thread fertig ist
    Console.WriteLine("---Ende---");
    Console.ReadKey();
}
```

```
private static void MachEtwas()
{
    for (int i = 0; i < 50; i++)
    {
        Thread.Sleep(100);
        Console.Write("zZZZ...");
    }
}
```

Thread beenden

```
public static void Main(string[] args)
{
    Thread t = new Thread(MachEtwas);
    t.Start();

    Thread.Sleep(3000);
    t.Abort();
    Console.ReadKey();
}
```

```
private static void MachEtwas()
{
    for (int i = 0; i < 50; i++)
    {
        Thread.Sleep(100);
        Console.Write("zZZZ...");
    }
}
```

Threadpool

```
public static void Main(string[] args)
{
    ThreadPool.QueueUserWorkItem(MachWas);

    Console.WriteLine("---ENDE---");
    Console.ReadKey();
}

private static void MachWas(object arg1)
{
    // Logik
}
```

Vordergrund- und Hintergrundthreads

- Thread.IsBackground
- Unterschied zwischen Vordergrund- und Hintergrundthreads:
 - Solange Vordergrundthreads laufen, wird die Anwendung von der Runtime ausgeführt.
 - Wenn der letzte Vordergrundthread zu Ende ist, wird die Anwendung geschlossen und alle Hintergrundthreads gestoppt.

```
// Standardfall: Vordergrundthread  
Thread t1 = new Thread(MachEtwas);  
t1.Start();
```

```
// Standardfall: Hintergrundthread  
ThreadPool.QueueUserWorkItem(MachEtwasImThreadPool);
```


Synchronisation

- Problem: mehrere Threads greifen auf die selbe Variable zu
- Lösungen:
 - Monitor / lock() {}
 - Mutex
 - Interlocked
 - System.Collections.Concurrent
 - ...

Monitor / Lock()

- Monitor ermöglicht das synchronisieren eines Codeblocks
- Zur Synchronisation wird ein beliebiges Objekt benötigt, das immer nur von einem Thread gesperrt werden kann

```
private readonly object lockObject = new object();
private int betrag;

public void Erhöhen()
{
    Monitor.Enter(lockObject);
    betrag++;
    Monitor.Exit(lockObject);
}

public void Verringern()
{
    lock (lockObject)
    {
        betrag--;
    }
}
```

<https://coders-corner.net/2013/04/09/multithreading-in-c-teil-11-lock-monitor-mutex-semaphore-semaphoreslim/>

Mutex

- Mutex verhält sich wie ein Monitor, erlaubt aber die Verwendung über mehrere Prozesse hinweg und ist daher auch in mehreren Instanzen einer Applikation gemeinsam nutzbar
- Anwendungsfälle:
 - Anwendung nur einmalig starten
 - Inter-Prozess Kommunikation synchronisieren
 - Synchronisieren einer gemeinsamen Aufgabe zweier Prozesse
 - Synchronisation eines gemeinsamen Zugriffs auf Ressourcen aus mehreren Prozessen heraus

Mutex - Beispiel

```
static void Main(string[] args)
{
    Mutex mutex = new Mutex(false, "MeinMutex");
    for (int i = 0; i < 100; i++)
    {
        mutex.WaitOne();
        Thread.Sleep(100);
        Console.WriteLine(i);
        mutex.ReleaseMutex();
    }
    Console.ReadKey();
}
```

Programm 1 mal starten -> schnell

Programm 3 mal starten -> langsamer da sich das auf die 3 aufteilt und jeder wartet bis er mal drann ist

Interlocked

- Stellt atomare Operationen zur Verfügung, die von mehreren Threads genutzt werden können
- Beispiele:
 - Add()
 - Increment()
 - Decrement()
 - Exchange()

Exchange => Wert auf was neues setzen und alten Wert zurückgeben

System.Collections.Concurrent

- Beinhaltet Thread-sichere Auflistungen
- Beispiele:
 - ConcurrentBag<T>
 - ConcurrentDictionary<T>
 - ConcurrentStack<T>
 - ConcurrentQueue<T>
 - SynchronizedCollection<T>
 - System.ServiceModel.Primitives